PORTFOLIO TECHNICAL REPORT

HSL Bus Reliability Analyzer

GTFS vs GTFS-RT Transit Performance Dashboard

Helsinki Region Transport (HSL) | Urban Mobility Data Science Project

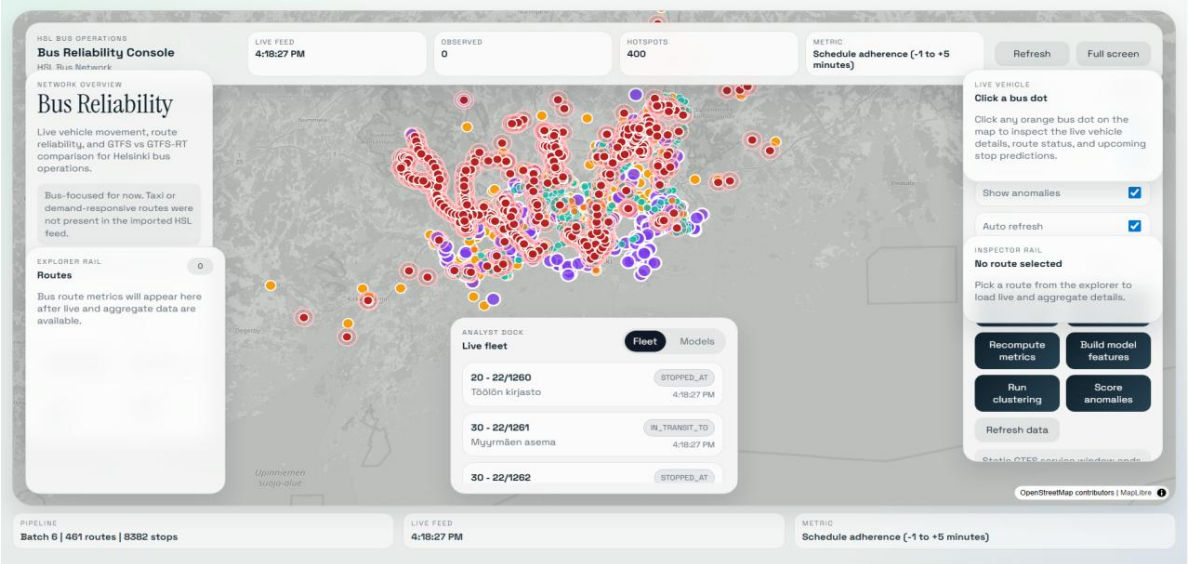


Figure: Live HSL bus reliability dashboard with GTFS-RT vehicle and model layers.

Project question: How reliably are HSL bus routes operating compared with planned schedules, and where should transport planners investigate delays, weak realtime coverage, or unusual service behavior?

Prepared as an illustrated project overview for portfolio, competition, and presentation use.

Executive Summary

The HSL Bus Reliability Analyzer is an end-to-end urban mobility data science project focused on Helsinki bus operations. It combines planned public transport schedules from static GTFS with live operational evidence from GTFS-RT. The project stores time-series and spatial data, transforms raw updates into model-ready route-stop-hour features, applies unsupervised learning, evaluates the resulting patterns, and communicates insights through an interactive map-first dashboard.

The official analysis scope is intentionally bus-only. The raw HSL feed contains multiple modes, but the modelling and dashboard focus on extended GTFS bus-family route types 701, 702, and 704. This keeps the reliability story consistent and avoids mixing bus behavior with rail, metro, tram, or ferry operations.

FEATURE WINDOWS	CLUSTER ASSIGNMENTS	DBSCAN HOTSPOTS	PREDICTION COVERAGE
164,538	**6,000**	**416**	**97.4%**

Snapshot note: These metrics come from the saved bus-only API snapshot generated on 2026-04-27T06:20:30.639Z. Values can change when the pipeline and model jobs run again.

What The Project Delivers

- A reproducible Docker Compose environment for database, backend API, GTFS-RT poller, and frontend dashboard.
- Static GTFS ingestion for planned routes, stops, trips, stop times, calendars, and route shapes.
- GTFS-RT background polling for live vehicle positions and trip updates.
- Bus-only feature engineering for schedule adherence, delays, prediction coverage, freshness, and spatial context.
- KMeans and DBSCAN clustering for broad reliability profiles and hotspot discovery.
- Isolation Forest and Local Outlier Factor (LOF) for unusual bus-operation windows.
- A React and MapLibre dashboard for live vehicle inspection, route comparison, model layers, and statistical analytics.

1. Problem Background And Objective

Transport planners need more than a timetable. They need to understand how planned bus service compares with real operations. Static schedules answer where and when buses should run. Realtime feeds show where vehicles are being reported and what arrival predictions are available. The project connects these two views to analyze schedule adherence and reliability.

> **Core objective:** Provide an explainable bus reliability workflow that helps identify reliable service patterns, delay-prone areas, low-coverage windows, and unusual operations using real HSL mobility data.

Why A Bus-Only Scope?

HSL operates a multimodal public transport system. For this project, the analysis is narrowed to buses because buses operate in road traffic and their reliability is directly connected to stop-level delays, route performance, prediction quality, and schedule adherence. The focused scope also makes model interpretation fairer and clearer.

Extended GTFS route_type	Meaning in this project	Reason included
701	Regional bus service	Bus-family reliability analysis
702	Express bus service	Bus-family reliability analysis
704	Local bus service	Bus-family reliability analysis

Important distinction: route_type is a transport mode/category code. It is not an individual bus line number such as 30, 52, 510, or 560.

Schedule Adherence Definition

The dashboard uses On-Time Performance (OTP) as its central reliability concept. A scored observation is considered on time when it arrives between one minute early and five minutes late relative to the schedule.

Status	Delay rule	Interpretation
Early	Earlier than -1 minute	Vehicle is ahead of the accepted schedule window
On time	-1 to +5 minutes	Vehicle is within the accepted operating window
Late	Later than +5 minutes	Vehicle is behind the accepted schedule window
Static only	No usable live comparison	Schedule exists, but realtime context is insufficient

2. Data Sources And Data Understanding

Static GTFS: The Planned Network

Static GTFS is the scheduled description of the public transport network. It is imported from the HSL GTFS package and provides the baseline against which realtime behavior is interpreted.

GTFS table	Purpose
routes.txt	Route identifiers, user-facing route names, and route types
stops.txt	Stop names and geographic coordinates
trips.txt	Scheduled service trips associated with routes
stop_times.txt	Scheduled stop arrival and departure times
shapes.txt	Route geometry used to draw map lines
calendar.txt / calendar_dates.txt	Service days and exceptions

GTFS-RT: The Operational Evidence

Realtime feed	Purpose
VehiclePositions	Where buses are currently reported on the map
TripUpdates	Predicted stop arrival/departure context and delay evidence
Service Alerts	Potential disruption notices; available as a future enhancement

Data Types

- Numerical: delay minutes, speed, coverage ratio, freshness, and vehicle observation count.
- Categorical: route id, stop id, route type, vehicle status, and model reliability label.
- Temporal: GTFS-RT recorded timestamps, scheduled times, hour of day, weekday, and weekend flag.
- Spatial: stop latitude/longitude, vehicle coordinates, and route geometry.

Key Data Challenges

- Realtime trip identifiers may be missing or may not cleanly match static GTFS trip identifiers.
- A missing TripUpdate does not automatically mean that a bus is on time.
- Prediction coverage varies by route and time window.
- Historical GTFS-RT accumulates only while the poller is running unless an external archive is used.

3. System Architecture

The system is modular: ingestion, storage, feature engineering, analytics, API delivery, and visualization are separated so each layer can be improved without rewriting the entire application.

1	**HSL Static GTFS**	planned routes, stops, trips, times, shapes
2	**GTFS-RT Poller**	vehicle positions and trip updates
3	**PostgreSQL + PostGIS + TimescaleDB**	relational, spatial, and time-series storage
4	**Feature Builder**	route-stop-hour model windows
5	**Models + FastAPI**	clusters, hotspots, anomalies, analytics endpoints
6	**React + MapLibre Dashboard**	live map and statistical intelligence

Architecture Explanation

Module	What it does	Why it matters
Static GTFS importer	Loads scheduled routes, stops, trips, stop times, shapes, and calendars	Provides the planned network
GTFS-RT poller	Fetches VehiclePositions and TripUpdates repeatedly	Stores realtime movement and predictions
PostgreSQL + PostGIS + TimescaleDB	Stores relational, spatial, and time-series records	Supports route, stop, map, and history analysis
Feature builder	Aggregates raw evidence into route-stop-hour windows	Creates machine-learning rows
Model jobs	Runs clustering and anomaly detection	Finds patterns, hotspots, and unusual windows
FastAPI backend	Serves ingestion status, live map layers, route metrics, and model outputs	Connects notebooks and dashboard
React + MapLibre frontend	Displays live operations and statistical analytics	Makes model evidence understandable

End-To-End Data Flow

1. Import static HSL GTFS to store planned bus routes, stops, trips, scheduled stop times, and route geometry.
2. Poll HSL GTFS-RT VehiclePositions and TripUpdates in the background.
3. Store raw realtime history in time-series tables while preserving spatial coordinates.
4. Filter analysis to bus-family route types 701, 702, and 704.
5. Aggregate matched evidence into route-stop-hour feature windows.
6. Run clustering and anomaly scoring jobs, then persist model outputs.
7. Serve the same evidence to notebooks and the React dashboard through FastAPI.

4. Preprocessing And Feature Engineering

Raw realtime feeds are not directly suitable for machine learning. The feature builder converts noisy event-level updates into route-stop-hour windows. A feature window is one summarized row for a particular bus route, stop, and hourly time bucket.

> **Simple explanation:** A feature window turns many raw bus updates into one compact row that describes how a route-stop-hour behaved.

Feature Window Contents

Feature group	Examples	Why it matters
Delay	Mean, median, p95 absolute delay, delay standard deviation	Measures reliability and variability
Adherence	On-time ratio, late ratio, early ratio	Shows schedule performance
Realtime quality	Prediction coverage ratio, feed age/freshness	Prevents missing data from being treated as good performance
Volume	Vehicle observation count	Shows how much evidence supports the window
Time	Hour, weekday, weekend flag	Captures daily mobility patterns
Spatial	Stop latitude and longitude	Supports maps and DBSCAN hotspot interpretation
Mode	Route type	Keeps analysis bus-focused

Missing-Value Handling

Missing realtime information is handled cautiously. Missing prediction data is not silently converted to zero delay. Instead, the project uses coverage and freshness features and can classify a location as static-only or low-realtime-coverage. This avoids the false conclusion that every vehicle with incomplete realtime data is on time.

5. Part 3: Data Mining Techniques

The course requires at least two data mining techniques. This project implements two categories: clustering/pattern discovery and anomaly detection. Each category uses two algorithms so their behavior can be compared.

KMeans: Broad Reliability Profiles

KMeans divides feature windows into a selected number of groups. It is useful for reducing thousands of bus route-stop-hour windows into a small number of understandable reliability profiles.

- Selected latest snapshot parameters: k = 2 and n_init = 20.
- Interpretation: the model creates broad groups such as more reliable behavior and windows requiring attention.
- Evaluation: silhouette = 0.681, Davies-Bouldin = 0.557, Calinski-Harabasz = 9,292.197.

DBSCAN: Hotspot Discovery

DBSCAN groups dense neighborhoods of similar points and leaves sparse points outside clusters as noise. It is useful because reliability problems may concentrate around corridors, terminals, or groups of stops rather than forming equal-sized clusters.

- Selected latest snapshot parameters: eps = 0.4 and min_samples = 15.
- Evaluation: silhouette = 0.621, noise share = 0.045, and cluster count = 4.
- Operational output: 416 saved DBSCAN hotspot assignments in the latest snapshot.

Isolation Forest: Global Unusual Windows

Isolation Forest identifies observations that are easier to separate from the main population. It is suitable for unusual delays, low prediction coverage, stale feed windows, or combinations of abnormal behavior.

- Selected latest snapshot parameters: n_estimators = 100, max_samples = 256, contamination = 0.05.
- Evaluation: Precision@K = 0.508, recall = 0.506, false-alarm share = 0.025.

Local Outlier Factor: Local Deviations

LOF compares each feature window with its local neighbors. This is useful because a value that is ordinary for one route or time period may be unusual in another context.

- Selected latest snapshot parameters: n_neighbors = 10 and contamination = 0.02.
- Evaluation: Precision@K = 0.141, recall = 0.056, false-alarm share = 0.017.

Model interpretation: Clustering explains recurring reliability patterns. Anomaly detection creates a watchlist of unusual windows. An anomaly is a planner investigation signal, not a confirmed disruption.

6. Part 4: Model Evaluation And Interpretation

Why Internal Clustering Metrics Are Used

There are no official ground-truth cluster labels for HSL bus reliability. Therefore, clustering is evaluated with internal metrics that measure separation, compactness, and structure.

Metric	Better direction	Meaning
Silhouette Score	Higher	Clusters are more clearly separated with less overlap
Davies-Bouldin Index	Lower	Clusters are more compact and distinct
Calinski-Harabasz Score	Higher	Between-cluster separation is stronger relative to within-cluster spread

Why Pseudo-Labels Are Used For Anomalies

There is no complete official label set saying which bus-operation windows are truly anomalous. The project therefore uses operational pseudo-labels such as extreme delays, stale feeds, low prediction coverage, and late-heavy windows. These provide a practical evaluation baseline while remaining an explicitly stated limitation.

Metric	Meaning
Precision@K	How many top-ranked alerts overlap with operational pseudo-labels
Recall	How much pseudo-labeled suspicious behavior is captured
F1 proxy	Balance between precision and recall
False-alarm share	Share of alerts that may not overlap with operational triggers

Honest interpretation: Model metrics support prioritization and comparison. They do not prove the external cause of a delay, which may require traffic, weather, roadworks, or service-alert context.

7. Interactive Dashboard Walkthrough

The React and MapLibre dashboard connects the data science workflow to a planner-facing interface. It separates live operational evidence from model interpretation while keeping both visible on a shared map.

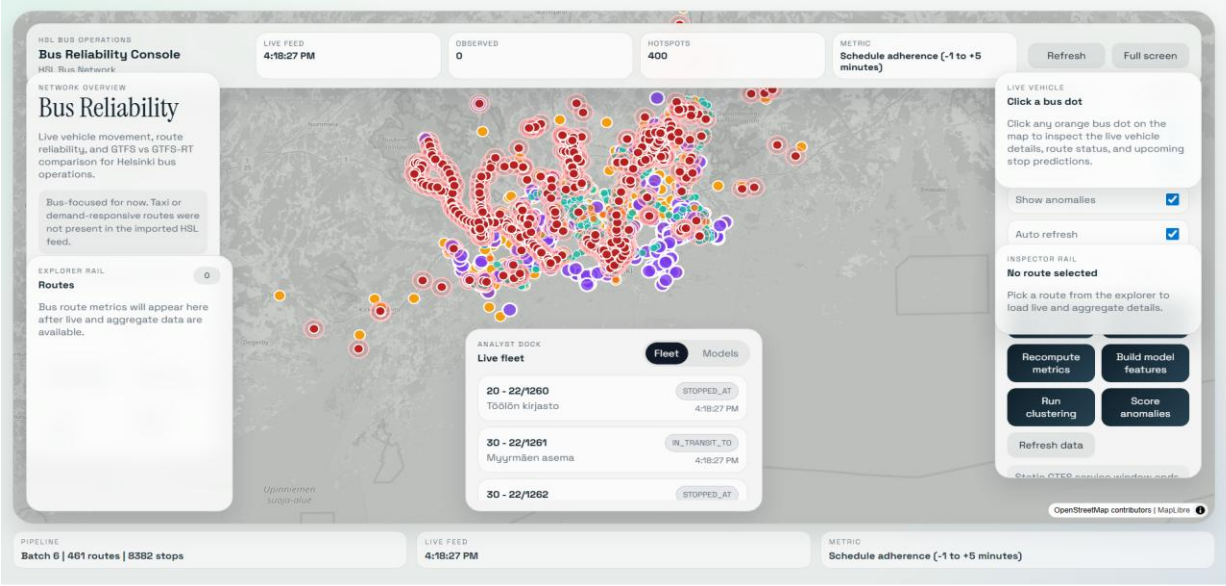


Figure: Live map view. Orange vehicle dots come from GTFS-RT; cluster and anomaly layers can be shown for investigation.

Map Layers And Colors

Color or visual	Meaning
Orange vehicle dots	Live GTFS-RT bus locations
Green markers	On-time stop comparison or reliable pattern
Red markers	Late stop comparison, poor reliability, or anomaly signal
Blue markers	Early stop comparison
Grey markers	Static-only schedule context without sufficient realtime comparison
Purple cluster markers	Low realtime prediction coverage
Teal cluster markers	Reliable or default cluster pattern
Strong route line	Selected static GTFS route geometry
Faint network lines	Planned background bus network

Planner-Facing Dashboard Areas

Dashboard area	What it explains
Executive KPIs	Live buses, observed routes, prediction coverage, and anomaly watchlist
Selected route intelligence	Route OTP, average delay, live buses, hotspots, and route coverage

Dashboard area	What it explains
Route explorer	Bus routes that currently have live or model evidence
Analyst dock	Live vehicle details and Part 3 model evidence
Control center	Layer toggles plus GTFS import, poll, metric, feature, clustering, and anomaly jobs
Statistical intelligence	Route health, coverage, anomaly pressure, rankings, labels, and reasons

8. Statistical Analytics View

The statistical intelligence section translates model outputs into concise operational summaries suitable for a presentation or planner review.

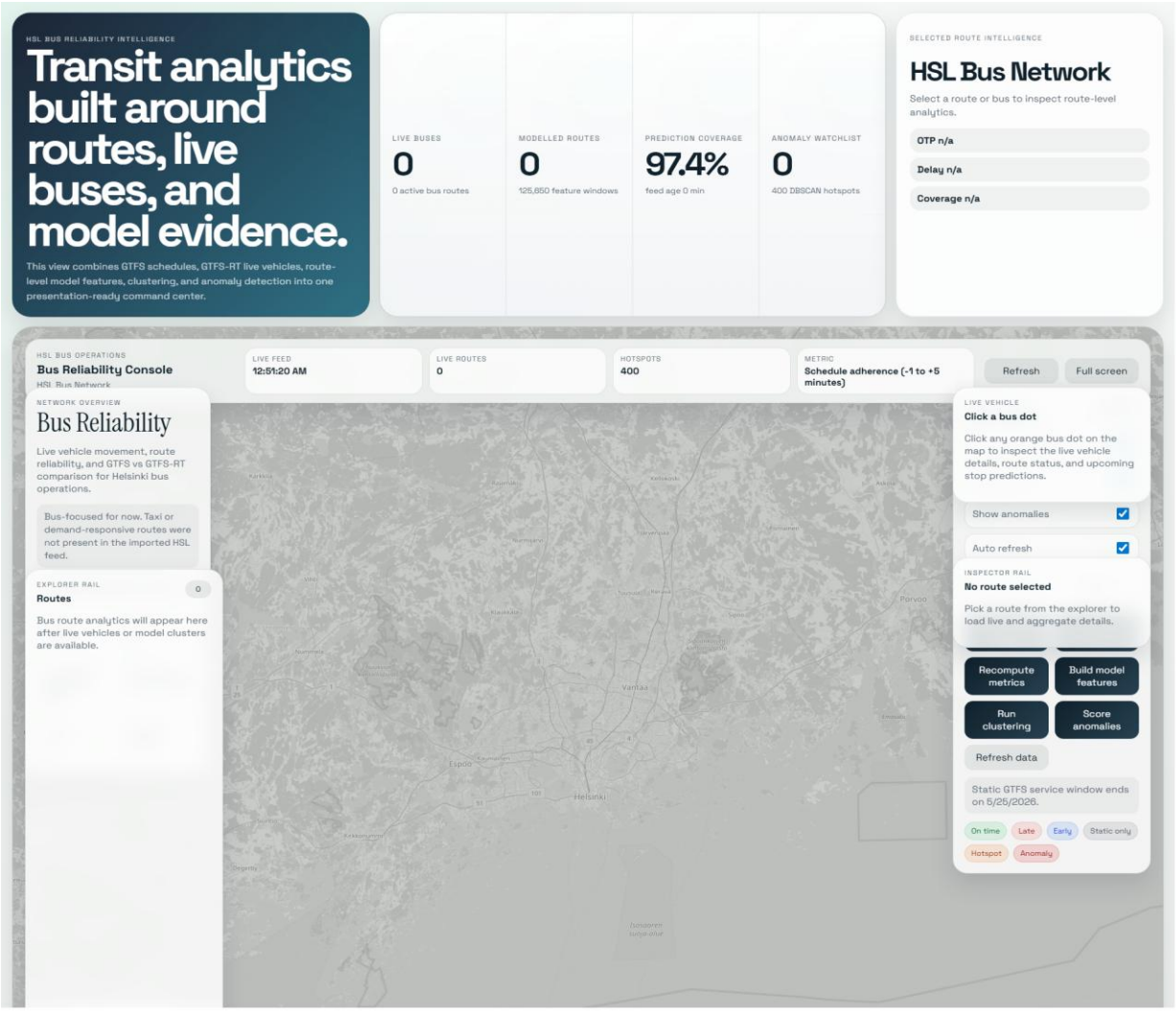


Figure: Industry-style dashboard layout with KPI cards, route intelligence, live-map stage, and control rail.

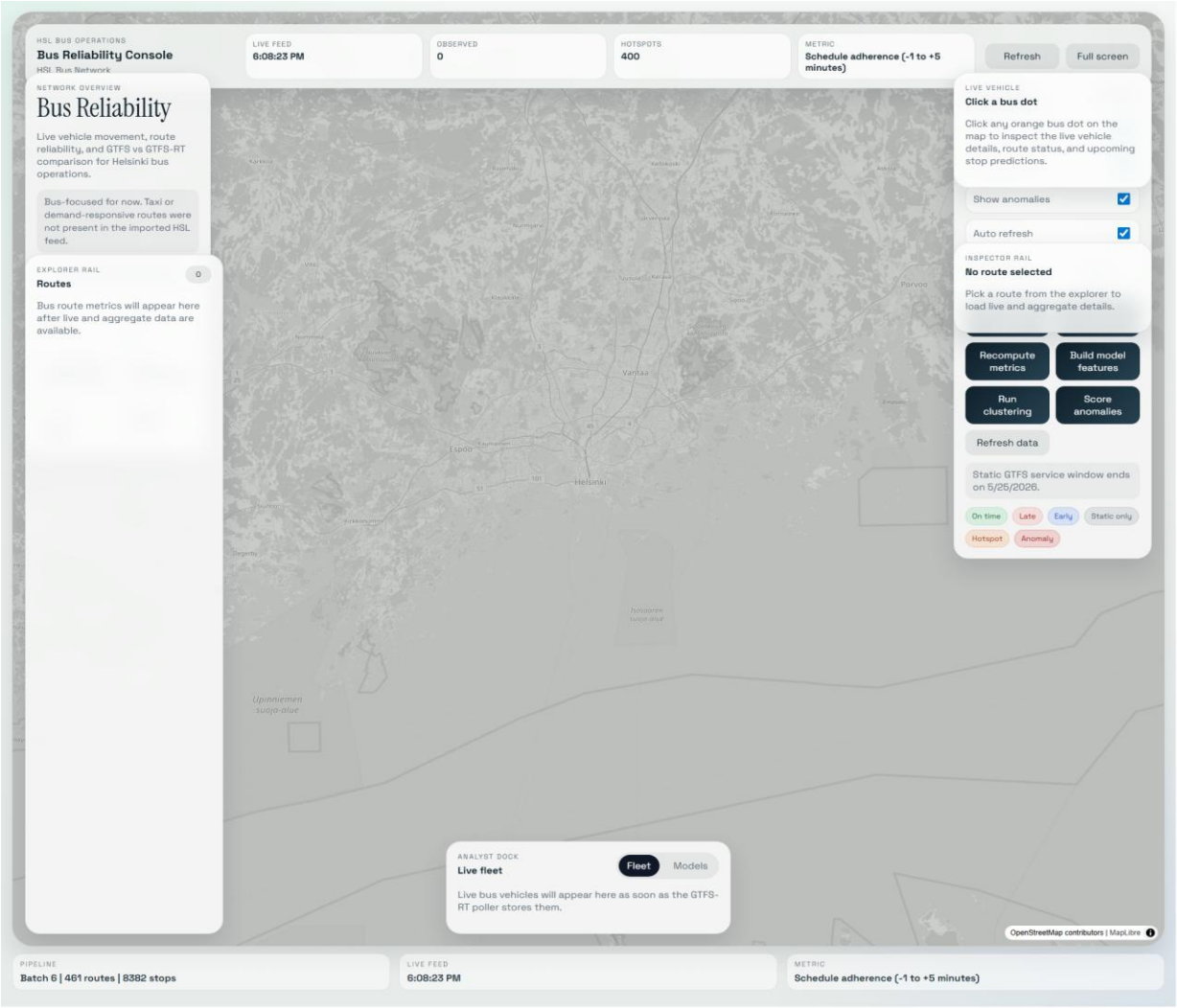


Figure: Map-first dashboard state showing the reliability legend and operational controls.

How To Explain The Analytics Cards

- Route health distribution groups OTP-scored routes into good, watch, and poor bands.
- Prediction coverage shows how much usable realtime signal exists for the feature windows.
- Live model watchlist shows anomaly pressure; it is a prioritization queue, not a confirmed incident list.
- Highest average delay ranks routes that deserve investigation.
- Vehicle status split summarizes buses reported as IN_TRANSIT_TO, INCOMING_AT, or STOPPED_AT.
- Reliability labels summarize recurring model patterns such as reliable, watch, delay_hotspot, and low_realtime_coverage.
- Anomaly reasons explain why unusual windows were flagged, for example extreme delay or coverage dropout.

9. Insights For Urban Mobility Planning

The project supports operational prioritization rather than automated decision-making. It helps planners decide where deeper investigation is most valuable.

- Use route health and average delay rankings to identify routes that may need timetable review or corridor-level investigation.
- Use DBSCAN hotspots to identify stop areas or corridors where reliability concerns concentrate.
- Use low-realtime-coverage labels to distinguish weak data quality from genuine service performance.
- Use anomaly windows as a focused watchlist for unusual operations requiring planner review.
- Use live vehicle popups and upcoming-stop context to connect network-level analytics back to individual bus movements.

10. Limitations And Future Work

Current limitation	Why it matters	Future improvement
Limited continuous GTFS-RT history	History accumulates only while the local poller runs	Deploy a persistent cloud poller and retention policy
No perfect GTFS-to-GTFS-RT match	Trip identifiers can be missing or noisy	Improve fuzzy linking with shape progress and stop sequence context
Pseudo-label anomaly evaluation	Operational triggers are not absolute ground truth	Add planner-reviewed labels and service-alert overlap
Bus-only analysis	Other HSL modes are excluded intentionally	Add mode-specific models without mixing behavior
No supervised ETA model	Project focuses on required unsupervised techniques	Train delay or ETA prediction after collecting longer history
Local dashboard deployment	Current Docker stack is not yet public-hosted	Deploy behind HTTPS with restricted admin operations

11. Reproducibility And Evidence

The project stores both human-readable and machine-readable evidence so the analysis can be reproduced and presented even when the live poller is not running.

Main Evidence Files

- notebooks/03_data_mining_gtfs_rt.ipynb - clustering and anomaly detection workflow.
- notebooks/04_model_evaluation_interpretation.ipynb - metrics, visualizations, and interpretation.
- reports/snapshots/bus_model_snapshot_latest.json - detailed saved API/model snapshot.
- reports/snapshots/bus_model_snapshot_compact.json - compact saved snapshot.
- reports/history_exports/ - recent bus GTFS-RT and model CSV exports.
- reports/screenshots/ - dashboard evidence screenshots.
- docs/dashboard-readme.md - dashboard teaching guide.
- docs/code-walkthrough-for-presentation.md - code and architecture explanation.

How To Run Locally

Start Docker Desktop, then run: `docker compose up -d`

Dashboard: `http://localhost:5173/`

FastAPI docs: `http://localhost:8000/docs`

Stop safely without deleting stored data: `docker compose down`

> **Data retention warning:** Do not use docker compose down -v unless you intentionally want to delete the database volume and collected GTFS-RT history.

12. Competition And Portfolio Summary

This project demonstrates a complete applied urban mobility data science workflow: open-data ingestion, spatial and time-series storage, careful preprocessing, feature engineering, unsupervised learning, quantitative evaluation, API design, and an explainable live dashboard.

> **One-sentence portfolio summary:** Built an end-to-end HSL bus reliability analyzer using static GTFS and GTFS-RT feeds, PostgreSQL/PostGIS/TimescaleDB, FastAPI, clustering, anomaly detection, and a React/MapLibre dashboard for live operations and planner-facing insights.

Suggested Closing Statement

The project turns raw HSL schedule and realtime feeds into interpretable reliability evidence. Static GTFS describes the planned network, GTFS-RT adds live operational context, the feature builder creates route-stop-hour model windows, clustering reveals recurring reliability patterns and hotspots, anomaly detection prioritizes unusual behavior, and the dashboard makes the results understandable for transport planners.

Appendix: Five-Minute Demo Sequence

1. Open the dashboard and explain the four executive KPIs.
2. Select a route and show the static route geometry against live orange bus dots.
3. Click a bus dot to explain vehicle, trip, next-stop, speed, status, and update context.
4. Enable clusters and explain reliable, watch, delay_hotspot, and low_realtime_coverage labels.
5. Enable anomalies and explain that red signals form an investigation watchlist.
6. Scroll to the statistical analytics section and summarize route health, coverage, delay ranking, and anomaly reasons.